

איך צוות ה-AI שלי כותב קוד

בינה מלאכותית אחת חכמה הופכת לצוות שלם: בוס שחושב, עוזרים שעושים את העבודה השחורה, מבקר שמחפש טעויות, ושומרים שעוצרים תאונות גדולות.

המוטו: ידיים זולות, עיניים יקרות.

עוזרים מהירים וזולים עושים את הריצות. המוח הכי חכם שמור לחשיבה, להחלטות ולבדיקה.

קודם כל, איזו בעיה זה פותר?

Claude Code הוא AI שיועד לקרוא ולכתוב קוד. אם נותנים ל-AI אחד לעשות הכול לבד, קורים שני דברים רעים:

- זה יקר.** המודל הכי חכם הוא גם הכי יקר. להשתמש בו כדי לקרוא מאות קבצים זה כמו להזמין מנתח מומחה כדי שיטאטא את הרצפה בבית החולים.
- הזיכרון שלו מתמלא.** AI יכול להחזיק רק כמות מוגבלת של טקסט בראש בבת אחת. לזה קוראים **קונטקסט (context)**, והוא נמדד ב**טוקנים** (חתיכות קטנות של מילים). כשהוא מתמלא מדי, ה-AI נהיה איטי ומתחיל לשכוח פרטים.

הפתרון: לעבוד כמו מטבח של מסעדה

במסעדה טובה השף הראשי לא חותך כל בצל לבד. השף מתכנן את התפריט, נותן הוראות ברורות, ו**טועם כל מנה לפני שהיא יוצאת מהמטבח**. העוזרים במטבח חותכים, מערבבים ומוודים זמנים.

ה-Coding Orchestrator הוא אוסף של **חוקים, עוזרים וכלי בטיחות** שגורם ל-Claude לעבוד בדיוק ככה. ה"אורקסטרטור" הוא השף הראשי (כמו מנצח של תזמורת: הוא לא מנגן בכל כלי, אבל שומר שכולם ינגנו יחד).



הכירו את הצוות

כל חבר צוות הוא AI עם תפקיד אחד. הם משתמשים ב**מודלים** שונים ("מוחות" שונים): מוח גדול חושב טוב יותר אבל עולה יותר, אז כל תפקיד מקבל בדיוק את המוח שהוא צריך.

השף הראשי • האורקסטרטור • Opus 5.5 (החושב הגדול) מתכנן, מחליט ובודק הכול

מדבר איתכם, מבין מה אתם באמת צריכים, מפרק את העבודה למשימות קטנות וכותב לכל עוזר הוראות ברורות (**תדריך**). הוא מקבל את ההחלטות הקשות: איך לתכנן, מה הסיבה האמיתית לבאג, וכל דבר שקשור לאבטחה. והוא **בודק את העבודה של העוזרים** לפני שהוא אומר לכם שסיימנו.

סייר (Scout) • Sonnet (מהיר וזול) החוקר

מוצא קבצים, קורא קוד ומחזיר סיכום קצר. הוא רק **קורא** ואף פעם לא משנה כלום. ככה השף לא צריך למלא את הזיכרון שלו בקבצים שלמים.

מריץ (Runner) • Sonnet (מהיר וזול) הבודק

מריץ בדיוק את הפקודה שקיבל (למשל "תריץ את כל הבדיקות") ומדווח מה קרה: עבר או נכשל, ומה השגיאה המדויקת. הוא לא מנסה לתקן. פלט של בדיקות יכול להיות אלפי שורות, והמריץ מקצר אותו.

בנאי (Builder) • Sonnet (מהיר וזול) המבצע

מבצע שינוי **שכבר הוחלט עליו** ("להוסיף את הכפתור הזה בקובץ הזה"), ואז מריץ בדיקה שמוכיחה שזה עובד. החשיבה על **מה** לבנות כבר נעשתה אצל השף. הבנאי אף פעם לא עושה `push` או `commit`.

מבקר (Critic) • Fable (עין חדה) מוצא התקלות

התפקיד היחיד שלו הוא למצוא מה **לא בסדר**. הוא מגיע "בעיניים רעננות": הוא לא רואה את ההסברים של השף, אז אי אפשר לשכנע אותו להסכים. קוראים לו לשינויים מסוכנים: אבטחה, מחיקת מידע, דברים שתוכנות אחרות תלויות בהם.

משימות קטנות מדלגות על הצוות. אם משהו לוקח בערך שלושה צעדים או פחות, השף פשוט עושה אותו. לכתוב תדריך לעוזר היה לוקח יותר זמן מהמשימה עצמה. (אבל משימה קטנה ומסוכנת עדיין עוברת אצל המבקר.)

מי עושה מה, ועם איזה מוח

חוק הזהב לגבי פעולות גדולות

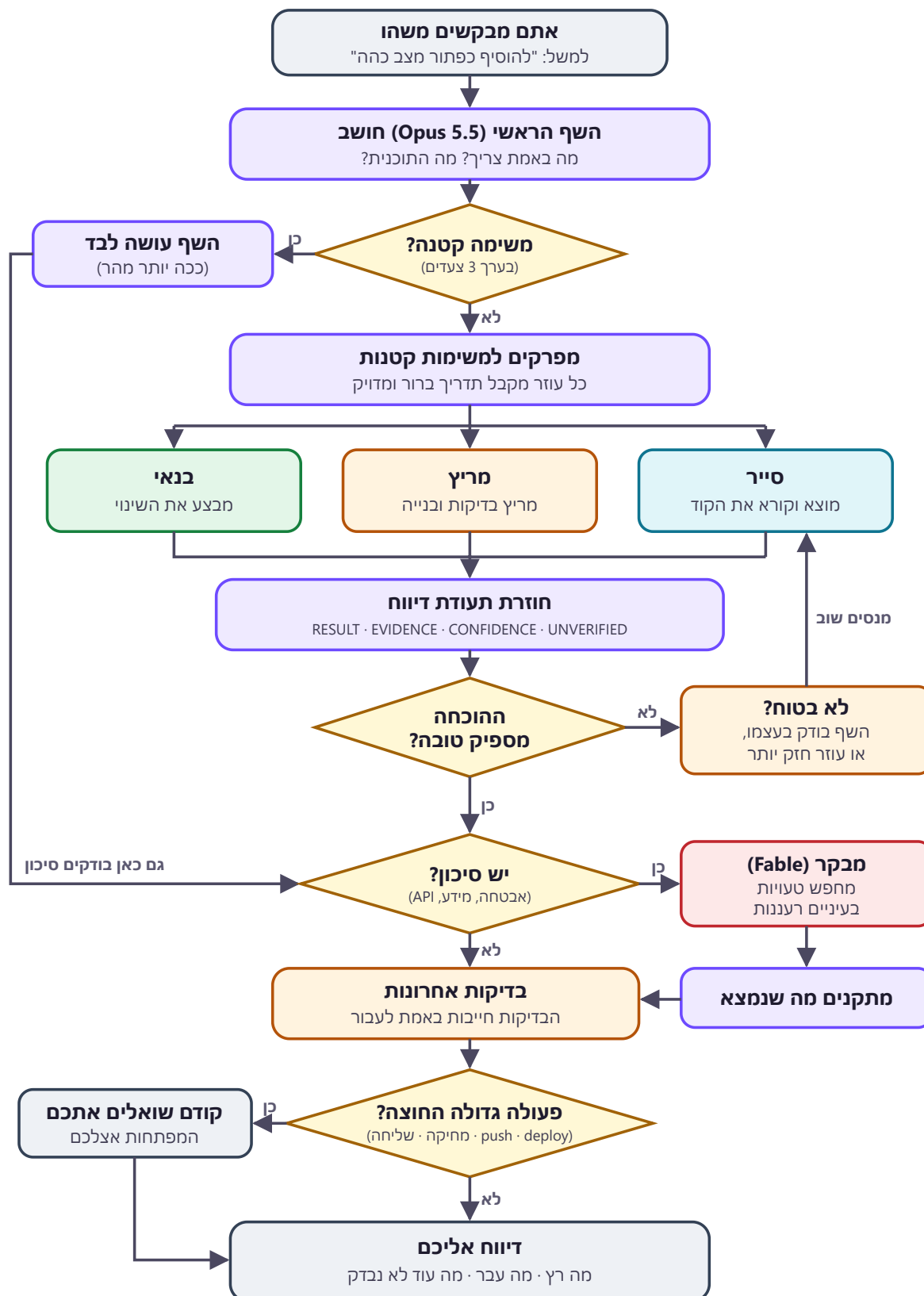
יש פעולות שיוצאות מהמחשב שלכם וקשה לבטל אותן: **push** של קוד ל-GitHub, **פרסום**, **העלאה לאוויר** של אתר (deploy), **מחיקה**, **ושליחת** הודעות. לעוזרים אסור לעשות את זה. רק השף יכול, ורק אחרי שאתם אישרתם.

איזה מודל עושה מה

תפקיד	Claude Code	Codex	למה דווקא הוא
השף	Opus 5.5	GPT-6.1 Sol · medium	החלטות קשות ובדיקה סופית צריכות את המוח הכי חזק.
סייר	Sonnet	GPT-6 Luna · low	לקרוא ולסכם: מהיר וזול זה מספיק.
מריץ	Sonnet	GPT-6 Luna · low	רק מריץ פקודה ומדווח מה יצא.
בנאי	Sonnet	GPT-6.1 Sol · medium	כותב קוד לפי תוכנית שכבר הוחלטה.
מבקר	Fable	GPT-6 Astra · high	מוח אחר וחד, שטוב במציאת טעויות.
Jev	jev-latest (TypeSafe)		שופט קטן ומהיר לשאלות קצרות ולציונים. כשהמשימה קשה, הוא יכול להעביר עוזר מ-Sonnet למוח גדול יותר: Opus או Fable.

תרשים 1: החיים של בקשה

ככה זה עובד מהרגע שאתם מבקשים משהו ועד שאתם מקבלים תשובה. המעוינים הם שאלות: עקבו אחרי החץ של "כן" או של "לא".



סגול = השף · כחול/כתום/ירוק = העוזרים · אדום = המבקר · מעוינים צהובים = שאלות

דוגמה אמיתית, צעד אחרי צעד

נניח שביקשתם: "להוסיף לאפליקציה שלי כפתור מצב כהה."

1. **השף חושב.** איפה מוגדרים הצבעים באפליקציה? יש כבר מסך הגדרות? זה יותר משלושה צעדים, אז הוא מפרק את העבודה.
2. **הסייר יוצא לחפש.** התדריך: "תמצא איפה מוגדרים הצבעים ומסך ההגדרות. אל תשנה כלום." הסייר חוזר עם: "הצבעים ב- `theme.dart` בשורות 10–40; מסך ההגדרות ב- `settings.dart`."
3. **השף מתכנן את השינוי** לפי הסיכום הזה, בלי לקרוא בעצמו כל קובץ.
4. **הבנאי מבצע.** התדריך: "תוסיף ערכת צבעים כהה ב- `theme.dart` ומתג ב- `settings.dart`. אחר כך תריץ את בדיקות ההגדרות." הבנאי מחזיר תעודת דיווח **וגם** את השינוי עצמו (שנקרא **diff**), והשף קורא אותו.
5. **המריץ מריץ את כל הבדיקות** ומדווח: "312 עברו, 0 נכשלו, קוד יציאה 0."
6. **יש סיכון?** מתג צבעים זה לא שינוי אבטחה, אז הפעם לא צריך מבקר.
7. **השף מדווח לכם:** מה השתנה, אילו בדיקות רצו ועברו, ומה הוא לא הצליח לבדוק (למשל "לא בדקתי על טלפון אמיתי").

תעודת הדיווח

כל עוזר חייב לענות עם אותם ארבעה חלקים. זה כמו להראות את הדרך בשיעור מתמטיקה: תשובה בלי דרך מקבלת פחות נקודות.

UNVERIFIED	CONFIDENCE	EVIDENCE	RESULT
מה הוא <u>לא</u> בדק. "לא בדקתי באייפון."	כמה הוא בטוח: גבוה, בינוני או נמוך. "גבוה."	הוכחה: אילו פקודות רצו ומה יצא. "בדיקות: 14 עברו, קוד יציאה 0."	מה קרה, בשורה או שתיים. "הוספתי מתג מצב כהה."

למה לא פשוט לסמוך על העוזרים?

לעוזרים יש מוחות זולים ומהירים יותר, אז לפעמים הם טועים או נשמעים בטוחים יותר ממה שהם באמת. לכן החוקים אומרים:

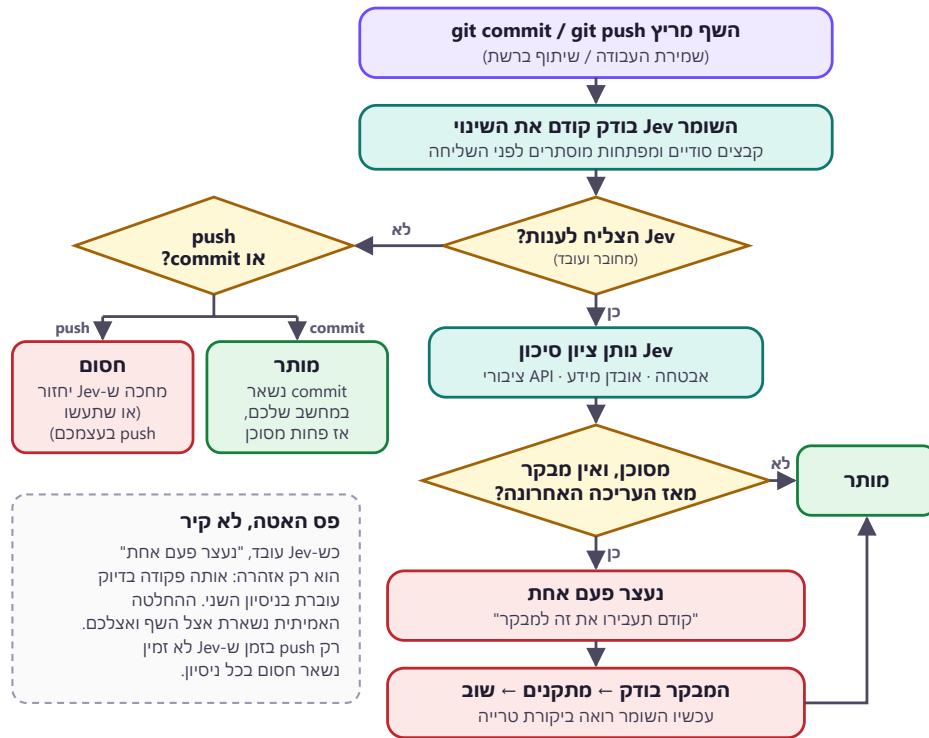
- אם **חסרה** הוכחה, או שהביטחון רק **נמוך או בינוני**, השף לא מבקש שוב מאותו עוזר. הוא **בודק בעצמו**, או **מעלה את המשימה רמה אחת** (סייר או מריץ ← בנאי ← השף).
- השף תמיד **קורא את השינוי האמיתי של הבנאי**, לא רק את הסיכום.
- את הבדיקות מריצים **אחרי העריכה האחרונה**. בדיקה שעברה לפני שינוי לא מוכיחה כלום על מה שאחריו.
- "נראה לי שזה עובד" זו לא תוצאה. תוצאה היא פקודה שרצה, יחד עם **קוד היציאה** שלה (0 אומר הצלחה; כל מספר אחר אומר שמשהו נכשל).

עובדה מעניינת: המבקר מקבל רק את השינוי ואת הטענה שהוא צריך לתקוף, אף פעם לא את המסקנות של השף. אם אומרים לבדוק "אני די בטוח שזה בסדר", הוא נוטה להסכים. עיניים רעננות תופסות יותר.

תרשים 2: שומר הבטיחות (Jev)

Jev הוא שופט AI קטן ומהיר מאוד מחברה בשם TypeSafe. זו תוספת אופציונלית (אצלנו היא דולקת). הוא אף פעם לא כותב קוד. הוא רק עונה בשבריר שנייה על שאלות קצרות כמו "כמה זה מסוכן?".

האורקסטרטור משתמש בו בכמה מקומות. החשוב ביותר הוא **שומר בדלת** לפני שקוד נשמר בהיסטוריה (`git commit`) או משותף ברשת (`git push`).



טורקיז = Jev · ירוק = מותר · אדום = נעצר

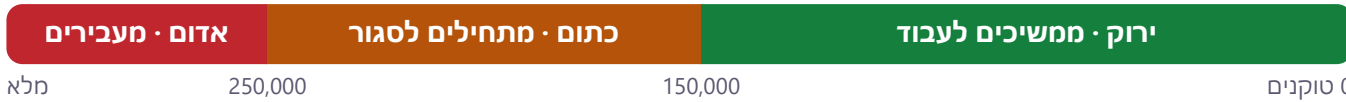
עוד תפקידים של Jev

תפקיד	מה Jev עושה
בוחר גודל מוח	לפני שעוזר מתחיל, Jev קורא את המשימה וממליץ איזה מודל יעשה אותה. הוא מעביר בקלות משימה למוח גדול יותר (לתת משימה קשה למוח קטן זו הטעות היקרה), אבל צריך להיות בטוח מאוד כדי להעביר משימה למוח קטן יותר. את המבקר הוא אף פעם לא מחליף.
בודק תעודות דיווח	אם בדיווח של עוזר חסר חלק, או שההוכחה לא תומכת בתוצאה, הדיווח נשלח חזרה פעם אחת לתיקון.
מזהה החלפת נושא	אם ההודעה החדשה שלכם על משהו אחר לגמרי, הוא אומר "תתחילו סשן חדש" (ראו בעמוד הבא).
נותן ציון לפתקי העברה	נותן ציון מ-0 עד 4 ל"מקל" (בעמוד הבא). פתק חלש לא נשמר עד שמתקנים אותו.
עונה על שאלות על קבצים	בעזרת ask-jev השף יכול לשאול "איזה מהקבצים האלה מטפל בכניסה למערכת?" בלי לקרוא את כולם לזיכרון שלו.

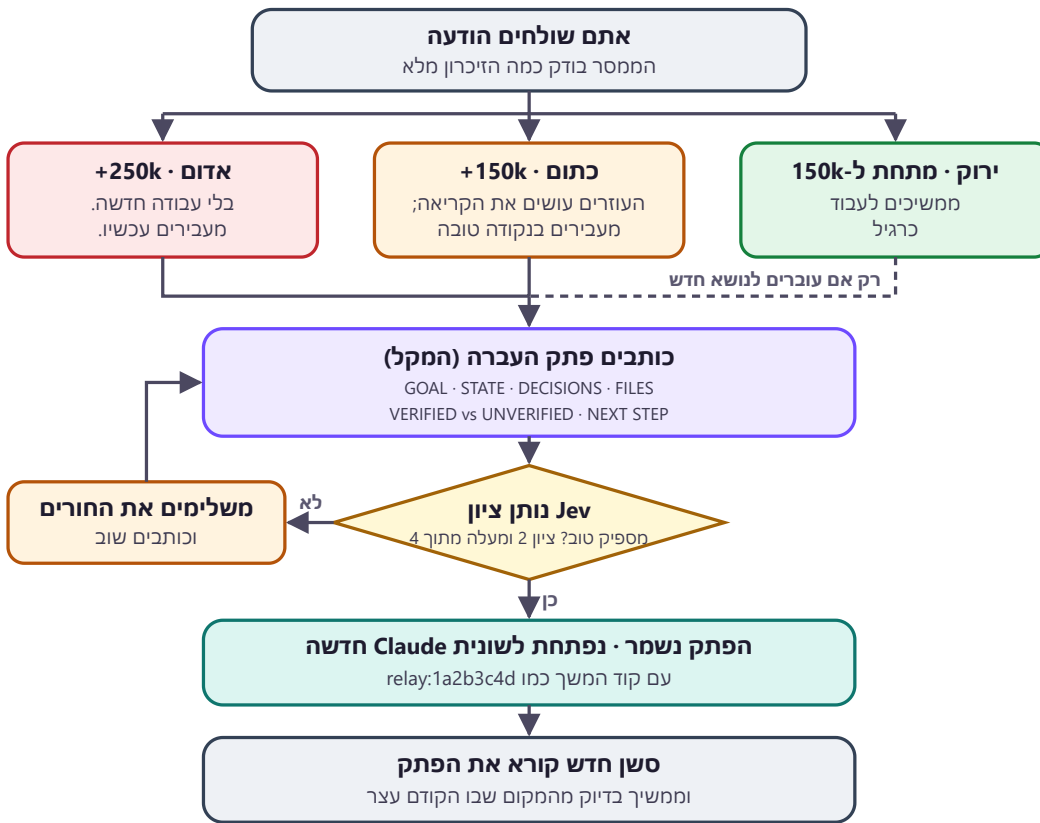
אם Jev לא זמין, כמעט הכול ממשיך לעבוד כמו בלעדיו ("נכשל פתוח"). יש חריג אחד: push של קוד לרשת חשוב מדי כדי להעביר אותו בלי בדיקה ("נכשל סגור").

תרשים 3: מרוץ שליחים (כשהזיכרון מתמלא)

זוכרים שלזיכרון של ה-AI (הקונטקסט) יש גבול? תוכנה קטנה שנקראת **המסר** (relay) בודקת כמה הוא מלא בכל פעם שאתם שולחים הודעה, ומציגה מד דלק:



כמו במרוץ שליחים: כשרץ אחד מתעייף, הוא מעביר את **המקל** לרץ רענן, שממשיך בדיוק מאותה נקודה. כאן המקל הוא **פתק העברה** כתוב (handoff).



מה כותבים בפתק ההעברה?

מה אנחנו מנסים להשיג.	GOAL (מטרה)
מה כבר נעשה ומה נשאר.	STATE (מצב)
מה הוחלט ולמה, וגם רעיונות שנפסלו (כדי שהסשן הבא לא ינסה אותם שוב).	DECISIONS (החלטות)
מה באמת נבדק (עם הפקודה והתוצאה) לעומת מה שרק מניחים. הסשן החדש בודק מחדש את מה שלא אומת.	VERIFIED vs UNVERIFIED
הדבר המדויק הבא שצריך לעשות.	NEXT STEP (הצעד הבא)

גם המסמך הזה נוצר אחרי העברה כזאת! הסשן הקודם התמלא, כתב פתק, וסשן חדש המשיך מאיפה שהוא עצר.

החוקים הגדולים, בעמוד אחד

- 1. **הבוס חושב, העוזרים מביאים.** תכנון, באגים מסובכים והחלטות אבטחה נשארים אצל השף. קריאה, הרצת בדיקות וביצוע הוראות ברורות עוברים לעוזרים זולים יותר.
- 2. **משימה קטנה? פשוט עושים.** אם זה בערך שלושה צעדים או פחות, לא צריך עוזרים.
- 3. **הוראות ברורות.** כל תדריך אומר מה המטרה, אילו קבצים בדיוק, מה החוקים, איך בודקים שזה עבד ומה להחזיר.
- 4. **מראים את הדרך.** בכל דיווח יש UNVERIFIED, RESULT, EVIDENCE, CONFIDENCE, עם קודי יציאה אמיתיים.
- 5. **לא סומכים, בודקים.** הוכחה חלשה? השף בודק בעצמו, או שהמשימה עולה רמה. אף פעם לא סתם "לנסות שוב את אותו דבר".
- 6. **מסוכן = מבקר.** אבטחה, מחיקת מידע ושינויים שתוכנות אחרות תלויות בהם עוברים ביקורת בעיניים רעננות לפני שהם נחשבים גמורים.
- 7. **המפתחות אצלכם.** push, פרסום, deploy, מחיקה ושליחה: רק השף עושה אותם, ורק באישור שלכם.
- 8. **מעבירים את המקל.** כשהזיכרון נגמר, כותבים פתק העברה טוב וממשיכים בסשן חדש.
- 9. **אומרים את האמת.** הדיווח הסופי אומר מה רץ, מה עבר ומה לא נבדק. כישלונות מדווחים מילה במילה.

מילים שכדאי להכיר

אורקסטרטור	ה-AI ה"בוס" שמתכנן ומתאם, כמו מנצח או שף ראשי.
סוכן משנה (Subagent)	AI עוזר שהאורקסטרטור מפעיל למשימה אחת (סייר, מריץ, בנאי, מבקר).
מודל	ה"מוח" של ה-AI. Opus 5.5 הוא החושב הגדול; Sonnet מהיר וזול; Fable הוא המוח של המבקר.
קונטקסט / טוקנים	הזיכרון לטווח קצר של ה-AI, וחתיכות המילים הקטנות שבהן מודדים אותו.
תדריך (Brief)	ההוראות שעוזר מקבל.
Diff	הרשימה המדויקת של השורות שנוספו או נמחקו בשינוי.
קוד יציאה	המספר שתוכנה מחזירה כשהיא מסיימת: 0 = הצלחה, כל מספר אחר = בעיה.
git commit / push	commit = שמירת "תמונה" של הקוד במחשב שלכם. push = העלאת התמונות האלה ל-GitHub בשביל אחרים.
Hook	תוכנה קטנה שרצה אוטומטית ברגעים מסוימים (למשל לפני שפקודה רצה). הממסר והשומר Jev הם hooks.
Jev	שופט AI מהיר (של TypeScript) שעונה על שאלות קצרות כמו "כמה מסוכן / איזה מהם / זה מספיק טוב?".
העברה / ממסר	הפתק והמערכת שמאפשרים לסשן חדש להמשיך את העבודה.